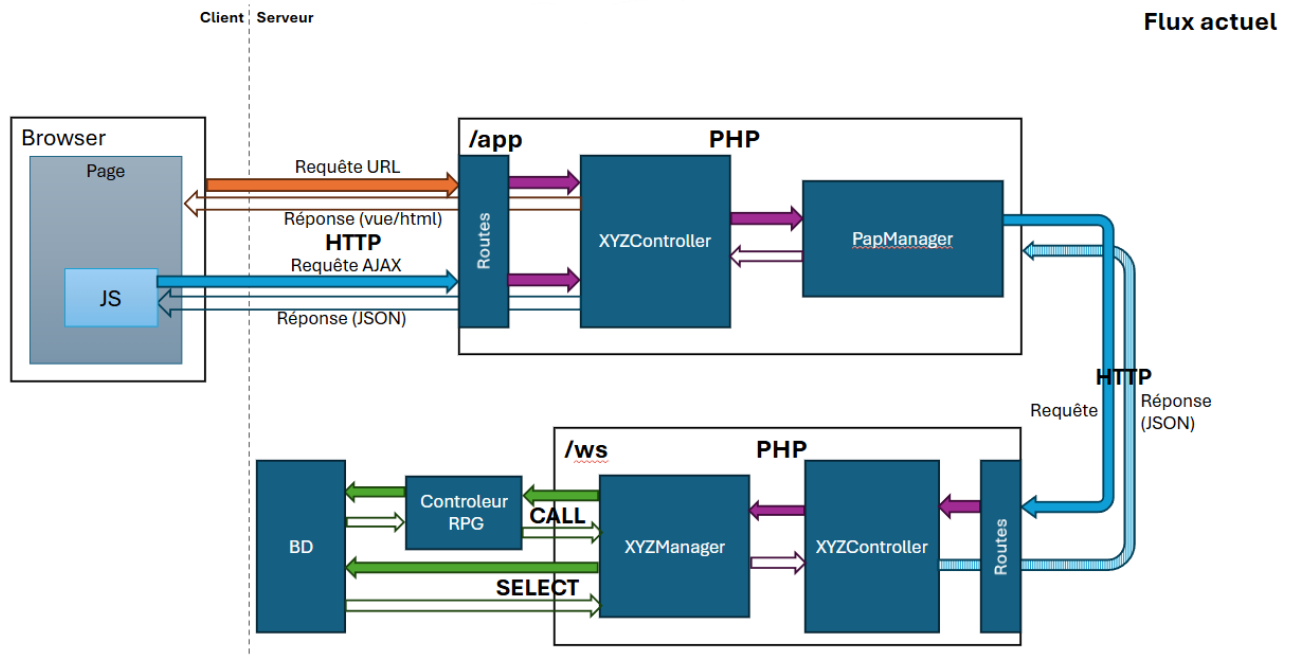


Stage

Table des matières

Ancien Plan.....	2
Problématique :.....	2
Plan Poc :.....	3
Outils/Language utiliser :	3
Git :.....	3
NPM:.....	3
IDE :.....	3
Postman :.....	4
Cree un environnement :.....	5
Base de données :.....	5
IbmDB2 :.....	5
Language:	6
Node.js:.....	6
Type Script:.....	6
JavaScript:	6
Framework :	6
NestJS:	6
Calendrier :.....	7
Semaine 1:	8
Semaine 2 :	8
Semaine 3 :	8
Semaine 4 :	8
Semaine 5 :	9
Problème :	9
Organisation du projet :.....	10
Les bases	10
Exemple : Controller	11
Exemple Service :.....	13

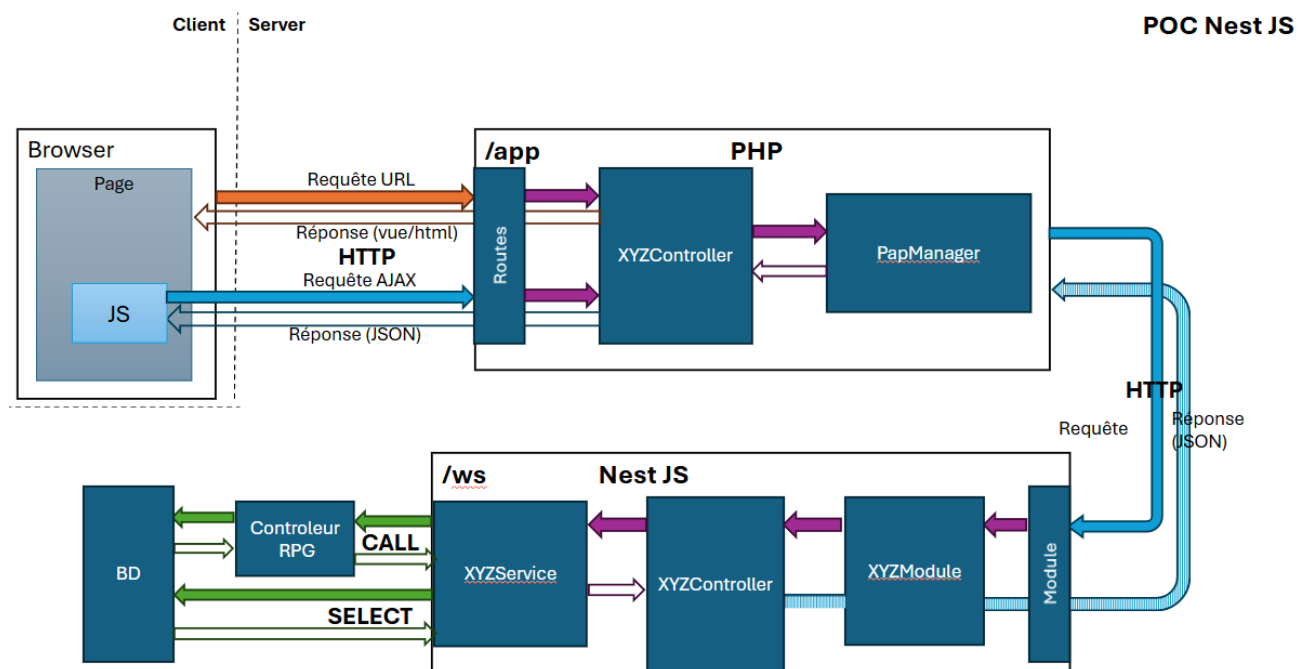
Ancien Plan



Problématique :

Faire un POC (Proof of Concept) sur le backend d'une application en utilisant le Framework Nest Pour permettre d'avoir un front et un back sur une même techno L'idée est de passer le backend de PHP à Node.js, pour moderniser les systèmes. Node.js est une technologie moderne, très utilisée, et elle est disponible nativement sur IBM.

Plan Poc :



Outils/Language utiliser :

Git :

Commande git de base

NPM:

Signifie Node Package Manager et est le gestionnaire de paquets par défaut pour l'environnement d'exécution JavaScript, Node.js. C'est un outil essentiel pour les développeurs JavaScript, leur permettant de partager et de gérer les paquets de code efficacement. Permis d'installer les packages et nodes nécessaires et de start le projet grâce à

`npm run start:dev`

IDE :

PHP Storm => environnement de travail : grâce au remote host dans PHP Storm on peut choisir un Virtual host et ainsi avoir un environnement de travail il suffit ensuite

de upload le code pour constater les modifications en testant l'app avec l'IP du Virtual host

Postman :

Utiliser pour envoyer des requêtes et tester les web service

Enregistrer les requêtes et crée des environnements de test

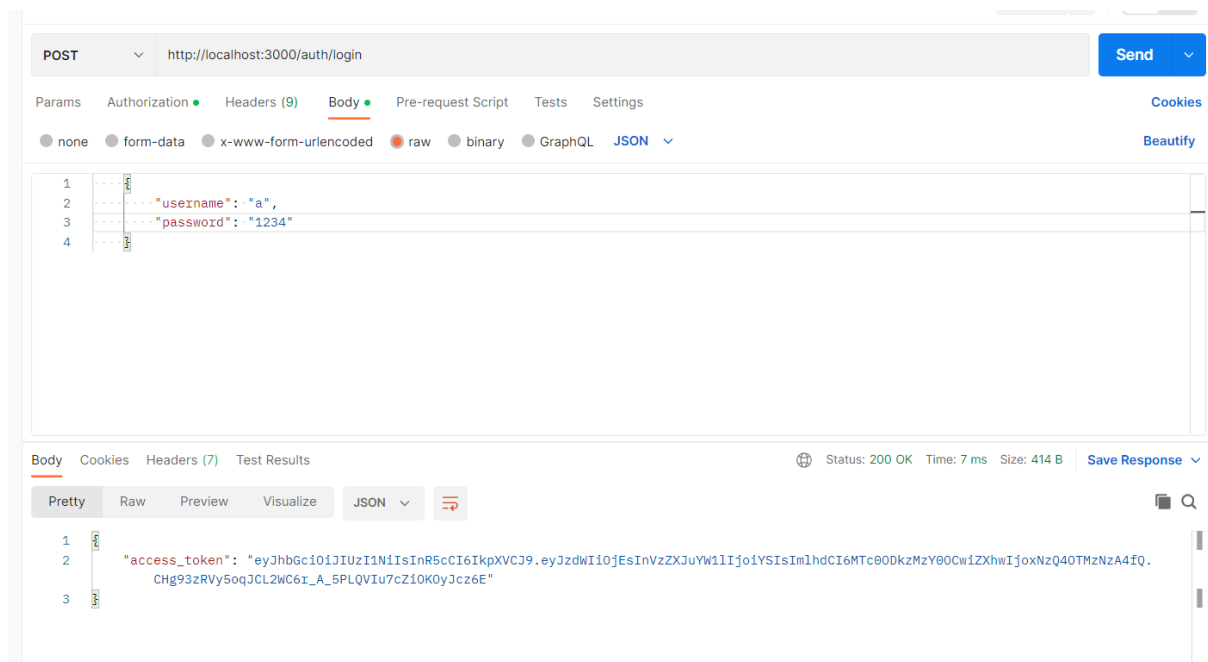
On peut mettre un body dans la requête pour tester des logins ou des post

Basic Auth => authentification classique

Token bearer coller le token renvoyer après le login

Tester api : ip dev+ chemin dans le mappage

Exemple d'utilisation : avec un login en post en local host



http://localhost:3000/ Save

GET http://localhost:3000/ Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

	KEY	VALUE	DESCRIPTION	...	Bulk Edit
	Key	Value	Description		

Body Cookies Headers (4) Test Results 200 OK 11 ms 154 B Save Response

Pretty Raw Preview Visualize Text

1 Voilà la réponse du serveur !

Cree un environnement :

develop Edit			
VARIABLE	INITIAL VALUE		CURRENT VALUE
ws	http://	/ws	http:// /ws

Base de données :

IbmDB2 :

IBM Db2 est une **base de données** offrant une évolutivité quasi infinie, des analyses en temps réel et un support multicloud

Module : (ibm_db n'as pas fonctionner) ODBC=> Fonctionne

Nom de la base egescome adutil (c'est le sablier pour lancer une requête)
mot de passe dans la table mais pas visible

Pour rajouter une liste de bibliothèque on peu rajouter ces deux paramètres
pour dans l'initialisation de la db ;NAM=1;DBQ=

Language:

Node.js:

Node.js est un environnement d'exécution open source et multiplateforme qui permet aux développeurs d'exécuter du code JavaScript en dehors d'un navigateur web, conçu pour créer des applications réseau évolutives.

Type Script:

TypeScript est un surensemble de JavaScript qui ajoute la typage statique optionnel et des fonctionnalités avancées à JavaScript

JavaScript:

JavaScript est un langage de programmation dynamique largement utilisé pour le développement web, permettant la création de pages web interactives et dynamiques.

Let=>la variable peut changer

Cons=>la variable est statique

Framework :

NestJS:

NestJS est un Framework permettant de créer des applications NodeJS côté serveur efficaces et évolutives. Et il utilise également le TypeScript.

Le principe est de découper son code et donc son application sous forme de modules. On injecte ensuite ces modules dans les parties de notre application où l'on en a besoin ils permettent de faire des liens entre leur service et leur middleware et ceux d'autre module comme par exemple avec le SqlService dans le module sql

Replace nestjs => npx @nestjs/cli puis la commande voulu
exemple : new project-name

Cree un controller: npx @nestjs/cli g co nomController

NestJs fonctionne avec 1 module ,1 controleur et 1service par api qui ont le même prefix et qui marche ensemble dans un dossier qui porte le nom du prefix

```
adutil.controller.spec.ts
adutil.controller.ts
adutil.module.ts
adutil.service.spec.ts
adutil.service.ts
```

AuthGuard permet de verrier token bearer

@query ⇔ \$_get[]

```
@Get( path: '...' )
getSupprimerFavoris(@Query()query:string[]){
    const user=(query['...']!=undefined ?query['...'] : "");
```

@Param('id') si dans la requête /:id et donc / 6...

```
@Get( path: '.../:...' )
getAgenceUser(@Param( property: '...' )user:string){
```

Scope.Request dans un @Injectable d'un Middleware ou d'un Service Permet d'exécuter le di service / Middleware à chaque requête HTTP, et

Donc en exécuter plusieurs en même temps

```
@Injectable( options: { scope: Scope.REQUEST })
```

Coté post avec @Post on peut récupérer des requêtes post et leur contenu grâce à @Body dans le cadre de mon stage les données transmises via des requête post étais dans un tableau en jison

```
@Post()
getListAdutil(@Body()...:string){
    return this.adutilService.listAdutil(...["..."])
```

Calendrier :

Semaine 1:

- Apprentissage de JavaScript/TypeScript/NodeJS.
- Essais de l'application Postman.
- Mise en place d'un environnement pour NestJS et essais du Framework.

Semaine 2 :

- Commencer le projet par l'authentification
- Index a la racine + de la doc json envoyer +login avec un authGuard qui vérifie si connecter
- Maîtrise complétée de Postman pour tester les api a dev et crée environnement
- Création de fonction pour générer et exécuter des requête SQL préparer

Semaine 3 :

- La base de données fonctionne le plus gros problème venait du module choisit en le changeant les problèmes de connexion à la base on était réglé
- Refaire la connexion/login en faisant connecter la personne a la base de données
- Finition des 3 api de job / 2 api d'agence / 4 api sql/ 1 api bid recette

Semaine 4 :

Trouver un moyen de dresser une liste de bibliothèque pour la bd

Ainsi qu'enregistrement de tous les tests sur Postman

Liste web service accomplie cet semaine

- Intitule= 2 web service
- Client = 4 Web Service
- Environnement=1 Web Service
- Favoris = 3 Web Service
- Logistique => Entrepôt=> = 6 Web Service
- Logistique => emballage = 4 Web Service
- Logistique => GestionDeStock = 9 Web Service

- Profil utilisateur = 2 Web Service
- Tache = 2 Web Service

Semaine 5 :

Finir les Get de Logistique et Client

Logistique => Transport = 5 Web Service

Connexion Front / Back

Finir le compte rendu / faire des tests

Mode de connexion :

auth basic => connexion normal => test connexion directement dans la base de données

+Scope.request pour pouvoir trier les connexion de chaque requête

Tokken bearer => vérification que tu es déjà connecter et te laisse passer => AuthGuard

Api key => le site pro a pro ping le back de Egescom avec cela a un moment surement plus maintenant

Problème :

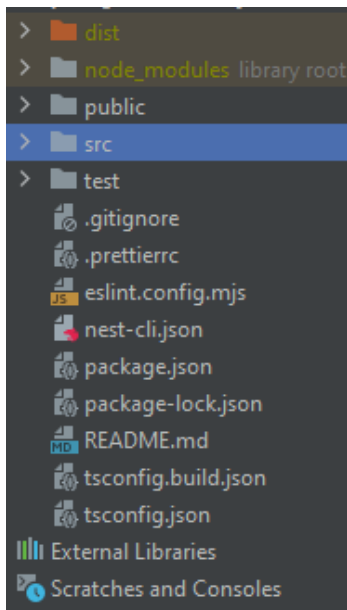
Connection base de données le module n'étais pas le bon => Utiliser odbc

Dresser une liste des bibliothèque /schéma pour que les requête s'exécute normalement

=> `;NAM=1;DBQ=`

Organisation du projet :

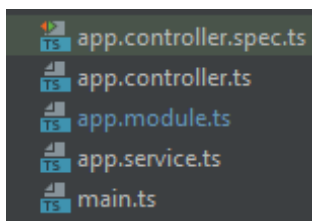
Les bases



Node_Modules => les module node qui ne sont pas transmit dans le git

Public : contient l'index et mon mapping

Src => contient le code



NestJs fonctionne de la façon suivante :

Étant à la « racine » du projet

On a donc 1 contrôler 1 service et 1 module app qui est créé dans le Main et qui est configurer dans celui-ci pour écouter sur le port 3000

```
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

1 usage  julien.bonnafe-ext
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen( port: process.env.PORT ?? 3000);
}

bootstrap();
```

```

import { Module } from '@nestjs/common';
import { SqlModule } from '../sql/sql.module';
import { AuthenticationModule } from '../authentication/authentication.module';
import { AdutilController } from './adutil.controller';
import { AdutilService } from './adutil.service';

2 usages  julien.bonnafe-ext
@Module({ metadata: {
  imports: [AuthenticationModule, SqlModule],
  controllers: [AdutilController],
  providers: [AdutilService],
}})
export class AdutilModule {}

```

Les Modules se présente de la façon suivante :

Imports : Permet de récupérer des service/MiddleWare d'autre module comme AuthGuard dans Authentification ou SqlService dans le module SQL.

Controllers : Permet liée le controller au Module

Providers : permet de liée le service

Au Module

Export : Facultatif permet d'exporter un service comme le SqlService du module SQL importé si dessus.

Exemple : Controller

Placeholder = anonymisation

```

import {Controller, Get, Param, Query, UseGuards} from '@nestjs/common';
import {IntituleService} from "../intitule.service";
import {AuthGuard} from "../authentication/auth.guard";

1 usage
@Controller()
export class IntituleController {
  no usages
  constructor(private intituleService: IntituleService) {
  }
  no usages
  @UseGuards(AuthGuard)
  @Get('intitules')
  getListIntitules(@Query() query: string[]) {
    const placeholder = (query['placeholder'] !== undefined ? query['placeholder'] : "");
    const placeholder1 = (query['placeholder1'] !== undefined ? query['placeholder1'] : "");
    const placeholder2 = (query['placeholder2'] !== undefined ? query['placeholder2'] : "");
    return this.intituleService.listIntitules(placeholder, placeholder1, placeholder2);
  }
  no usages
  @UseGuards(AuthGuard)
  @Get('intitule/:placeholder/:placeholder1')
  getSelectIntitule(@Param('placeholder') placeholder: string, @Param('placeholder1') placeholder1: string) {
    return this.intituleService.selectIntitule(placeholder, placeholder1);
  }
}

```

@controller => défini qu'il s'agit d'un contrôleur

Constructeur => construire le service permet de relier le service au Controller et donc d'utiliser ses fonctions

@get => il permet de préciser le type de requête traiter et de créer un path pour utiliser la fonction

@useguard permet de déclencher le authGuard du module authentication

: dans le path désigne que le param ('place Holder) récupère la donnée rentrer à la place du place Holder

@Query ⇔ \$_Get [] en php

@Param permet de récupérer la valeur rentrer dans le path de la requête

Exemple Service :

```
import {Get, Injectable} from '@nestjs/common';
import {IntituleController} from "../intitule.controller";
import {SqlService} from "../sql/sql.service";

1 usage
@Injectable()
export class IntituleService {
  no usages
  constructor(private sqlService:SqlService) {
  }
  no usages
  listIntitules(placeholder:string,placeholder1:string,placeholder2:string){
    let param=[placeholder];
    let sql="Requete SQL";
    if (placeholder!=""){
      sql+="Requete SQL";
      param.push(placeholder1,placeholder2);
    }
    return (this.sqlService.query(sql,param));
  }
  no usages
  selectIntitule(placeholder:string,placeholder1:string){
    let sql="Requete SQL";
    return this.sqlService.query(sql,[placeholder,placeholder1]);
  }
}
```

@Injectable => le décorateur est utilisé pour marquer une classe en tant que fournisseur, ce qui la rend disponible pour l'injection de dépendances dans toute l'application (middleware et Service)

Constructeur => permet d'utiliser le SQL Service qui est dans le module SQL

Exemple AuthGuard/Middleware

```
@Injectable()
export class AuthGuard implements CanActivate {
  //JWT pour le login et les verif de log in AuthService pour le logout
  no usages
  constructor(private jwtService: JwtService, private authService: AuthenticationService, private sqlService: SqlService) {}
  no usages
  async canActivate(context: ExecutionContext): Promise<boolean> {
    //on récupère la requête pour extraire le token du header si la var token est vide
    const request = context.switchToHttp().getRequest();
    const [type, token] = request.headers.authorization?.split(' ') ?? [];
    if(type==="Bearer"){
      //verif si la personne a un token et si il est valide
      if (!token || this.authService.isTokenBlacklisted(token)) {
        throw new UnauthorizedException( objectOrError: 'Token is invalid or blacklisted');
      }
      try {
        //test si le token est le bon
        const payload = await this.jwtService.verifyAsync(
          token,
          {
            options: {
              //secret: this.authService.getToken()
              secret: jwtConstants.secret
            }
          }
        );
      } catch {
        throw new UnauthorizedException();
      }
      //connecte
      return true;
    } else if(type==="Basic"){
      return this.tryConnect(token)
    }
    else{return false}
  }
}
```

```
//sépare le token du type et vérifie si c'est un token
//private extractTokenFromHeader(request: Request): string | undefined {
//const [type, token] = request.headers.authorization?.split(' ') ?? [];
// return type === 'Bearer' ? token : undefined;
// }
//teste de ce connecter a la base de donner avec les informations rentrer dans le auth basic
1 usage
private tryConnect(token: string): Promise<boolean> {
  console.log(token);
  const decryptLogin = Buffer.from(token, encoding: 'base64').toString( encoding: 'utf-8');
  console.log(decryptLogin);
  const [username, password] = decryptLogin.split( separator: ':');
  return this.sqlService.connect(username,password);
}
```

Middleware => En architecture informatique, un middleware est un logiciel tiers qui crée un réseau d'échange d'informations entre différentes applications informatiques

AuthGuard => Ils déterminent si une requête donnée sera traitée par le gestionnaire de route ou non dans le cas présent le authguard vérifie si quelqu'un se connecte avec un token si oui es que se token est conforme sinon on vérifie si la connexion n'est pas en en

basic auth et si oui l'authguard renvoie True si la tentative de connexion à la base de données fonctionne pour l'utilisateur

JWT => **JSON Web Tokens**

Dans une application **NestJS** c'est une approche courante pour la mise en œuvre de l'authentification et de l'autorisation. En effet dans le cas présent il est utilisé dans le cadre de l'autorisation pour permettre de vérifier si le token est conforme

Fonctionnement canActivate => est le « Main » de ce AuthGuard il prend en paramètre l'ExecutionContext et l'utilise pour récupérer le contenu de la requête

```
request = context.switchToHttp().getRequest()
```

Puis le split en un type et un token si le type est bearer on va vérifier s'il n'est pas sur notre liste noire puis on utilise JWT pour vérifier sa conformité si le type est Basic on va appeler tryConnect sinon on renvoie false

Fonctionnement tryConnect => va récupérer le token puis le décrypter et le split en login / mot de passe puis tenter une connexion à la DB renvoie True si ça marche False sinon